

A Dual-Loop Teacher-Anchored Residual Calibration Framework for Drift-Adaptive GKP Decoding

Abstract

Approximate Gottesman–Kitaev–Preskill (GKP) quantum error correction uses continuous modular syndromes to infer displacement errors, but finite-energy states and drifting operating conditions make a fixed correction law brittle. We study a dual-loop decoding framework that preserves a deterministic affine fast path, $\Delta_t = K_t s_t + b_t$, while using recent syndrome histograms to update the runtime parameters at a slower cadence. The design keeps the per-shot computation compatible with quantized fixed-point execution and makes the adaptation target explicit: learning or calibration acts only on a low-dimensional parameter surface, not on the full decoding decision.

A classical teacher estimator provides an interpretable anchor for the affine parameters, and a lightweight CNN predicts only a bounded residual calibration term, currently focused on the bias b_t . This differs from end-to-end neural decoders and from outer-code soft-information decoders: the learned module updates a physical-layer calibration surface, whereas the per-shot path remains a deterministic affine rule. The intended systems advantage is to reduce logical failures under drift while keeping the fast boundary close to a small fixed-point controller.

Under the controlled four-scenario simulation protocol, the teacher-anchored hybrid residual branch is the winner in all four scenarios, with UKF as the runner-up throughout; the UKF-relative mean reductions are 1.75%, 2.89%, 2.80% and 1.85% across the four reference rows, giving a 2.32% average reduction. The newer paired-interval source data strengthen two scenario rows: for `static.bias.theta` and `linear.ramp`, 12 paired repeats give positive UKF-minus-hybrid intervals, while the all-scenario, pooled, holdout, and hardware validations remain incomplete. Bounded ablation and multi-seed materials support only a descriptive reading of this result: histogram-delta features matter, the instability pattern repeats across seeds, and the tested lower-clip intervention is mixed and mostly harmful, so the current mechanism story remains non-causal. A supplementary statistical-calibration analysis further shows that a simple non-neural estimator can be strong under the same affine runtime contract, but the reported evidence does not select a unique promoted threshold. Controlled oracle-affine, wrapped-Gaussian, residual-channel surrogate, finite-squeezing toy-channel, holdout-stress, operation-count, fixed-point and runtime-counter checks add theory-facing diagnostics, not a new benchmark closure. The strongest current claim is therefore not that one CNN architecture closes the decoding problem, but that drift-adaptive affine calibration is a credible and hardware-bounded organizing principle for GKP fast-path decoding.

Contents

1	Introduction	2
2	Summary of Contributions	3
2.1	Metric-level advantages	4
3	Brief Review of the GKP Code	4
3.1	Ideal and approximate code states	5
3.2	Syndrome measurement as modular displacement information	5
3.3	Local affine decoding and its limits	6
3.4	Logical failure criterion	6

4	Noise and Drift Model	6
4.1	Effective noise state	6
4.2	Noise sources represented by the model	7
4.3	Drift scenarios	7
5	Model Architecture	7
5.1	Fast loop: affine fixed-point decoder	7
5.2	Parameter mapping	8
5.3	Teacher estimators	8
5.4	CNN residual branch	8
5.5	Statistical calibration branch	9
5.6	Stage-and-commit runtime contract	9
6	Relationship to Existing Work	9
6.1	GKP and bosonic analog soft information	9
6.2	Adaptive priors and syndrome-statistics estimation	9
6.3	Learned low-latency QEC modules	10
6.4	Real-time and FPGA QEC decoders	10
6.5	Advantages relative to existing QEC decoder approaches	10
7	Experimental Setup	11
7.1	Software-HIL protocol	11
7.2	Scenarios and modes	11
8	Numerical Results	11
8.1	Four-scenario affine benchmark	11
8.2	Descriptive UKF-versus-hybrid margins	12
8.3	Completed paired-interval scenario checks	12
8.4	Repeat-expansion protocol checks	12
8.5	Feature and teacher ablations	12
8.5.1	Supplement-side statistical calibration extension lane	13
8.6	Mechanism probe for residual-b behavior	13
8.7	Unseen drift generalization	13
8.8	Oracle and wrapped-Gaussian baselines	14
8.9	Runtime, quantization, and fixed-point degradation	14
8.10	Embedded runtime and board-level validation	15
8.11	Analysis-file coverage and statistical treatment	15
8.12	Hardware measurement requirements synchronized from the submission draft	15
9	Discussion	15
10	Conclusion	16

1 Introduction

Bosonic codes protect quantum information by encoding a logical qubit into the Hilbert space of an oscillator. The GKP code is a central example: ideal code states form a periodic lattice in phase space, so small shift errors can be diagnosed through modular syndrome measurements and corrected by displacement operations [1, 2]. Unlike binary stabilizer syndromes, GKP syndromes are analog. Their continuous values carry information not only about the likely displacement, but also about how close the correction is to a lattice decision boundary.

The practical difficulty is that physical GKP states are approximate finite-energy states. Their comb peaks have finite width and an envelope, and syndrome measurements are further affected by measurement inefficiency, noisy auxiliary states, circuit-level imperfections, oscillator loss, and calibration error [2, 9]. As a result, the effective noise model seen by the decoder can drift over time. A correction law tuned for one operating point may become systematically mismatched after bias, variance, or covariance changes.

Recent GKP and bosonic-decoding work makes clear that analog soft information is valuable. Surface-GKP and concatenated bosonic-QLDPC decoders use continuous syndrome information to set matching weights, belief-propagation messages, or effective outer-code priors [4, 5, 6, 7, 8]. In parallel, hardware-facing QEC studies show that learned modules are most useful when their latency role and integration boundary are explicit [15, 16], and calibration work emphasizes that decoder priors must remain aligned with measured device statistics [10, 11, 12, 13, 14]. These two lessons motivate the present formulation.

Our starting point is that drift adaptation should enter the GKP layer through a bounded runtime contract rather than a free-form per-shot decoder. The fast loop is intentionally simple:

$$\Delta_t = K_t s_t + b_t, \tag{1}$$

where s_t is the measured two-quadrature GKP syndrome and (K_t, b_t) are quantized runtime parameters. A slower loop observes recent syndrome histograms, estimates an effective noise state, and stages updated parameters into the runtime bank. The online correction path therefore remains a small affine operation, while nonstationary estimation is handled asynchronously through teacher-guided or calibration-driven updates.

This separation gives the method a specific systems role. Compared with a fixed affine decoder, the parameters can track drift. Compared with a full neural decoder, the per-shot computation remains smaller, more interpretable, and easier to validate in fixed-point form. Compared with outer-code soft-information decoders, the adaptation target is the physical-layer GKP displacement rule itself rather than a matching graph or LDPC message schedule. The learned branch is therefore not the real-time decoder; it is one possible slow-loop calibration module operating on a low-dimensional affine parameter surface.

This also positions the work relative to FPGA-QEC practice. Much of the real-time hardware literature focuses on decoding binary stabilizer syndromes for surface codes, repetition-code experiments, or quantum-LDPC memories using lookup tables, union-find variants, matching or greedy approximations, clustering, neural decoders, or message-passing hardware [18, 19, 20, 21, 22, 24, 25, 27]. Those results establish the standard for closed-loop latency and resource reporting, but they do not directly address a GKP physical correction layer whose measured syndrome is analog, modular, and shaped by finite-energy state preparation. The present architecture is therefore complementary: it provides a low-dimensional calibration layer that could sit before, inside, or alongside a larger stabilizer or LDPC decoding stack while keeping the fast boundary as a small fixed-point affine displacement.

The current note keeps the evidence hierarchy explicit. The controlled software-HIL benchmark is the main result layer. Feature and teacher ablations, six-seed mechanism materials, and the separately labeled statistical-calibration extension lane are supporting layers that help explain or contextualize the benchmark but do not replace it. Training/material regeneration, isolated true `.tflite` execution, and hardware-readiness artifacts remain boundary evidence rather than coequal performance results.

2 Summary of Contributions

The work can be organized around six bounded contributions.

- 1. A dual-loop affine calibration formulation for approximate GKP decoding.** The per-shot correction path is formulated as a bounded affine estimator $\Delta_t = K_t s_t + b_t$. The adaptation problem is moved into a slower loop that estimates an effective noise state from recent syndrome statistics and updates only the runtime parameters consumed by the fast path. This ties the continuous-syndrome GKP picture to a contract that remains compatible with quantized matrix-vector execution.

2. A teacher-anchored residual architecture. A classical teacher estimator produces a stable baseline $(K_t^{\text{teacher}}, b_t^{\text{teacher}})$, and the learned branch predicts only a bounded residual correction,

$$\delta b_t = f_\phi(H_{t-c+1:t}, \Delta H_t, z_t^{\text{teacher}}), \quad (2)$$

with committed parameters

$$K_t = K_t^{\text{teacher}}, \quad b_t = \text{EMA}\left(b_t^{\text{teacher}} + \delta b_t\right). \quad (3)$$

Learning is therefore used to recalibrate a low-dimensional control surface, not to replace GKP decoding end-to-end or to move a neural network onto the per-shot critical path.

3. A controlled reference evidence layer. The strongest performance evidence remains the controlled four-scenario software-HIL benchmark that serves as the reference comparison for the paper. Within that fixed set, the teacher-anchored hybrid residual branch is the winner in all four scenarios and UKF is the runner-up throughout; the scenario-wise UKF-relative reductions are 1.75%, 2.89%, 2.80% and 1.85%. This is the primary ranking result; it is not an expanded benchmark claim, not a deployment-closure result, and not a real-board result. The later paired-interval source data add scenario-level support for the direction of the UKF-vs-hybrid gap in `static_bias_theta` and `linear_ramp`, but they still do not complete the all-scenario repeat-expanded validation set.

4. Conservative supporting interpretation layers. Feature/teacher ablations and six-seed mechanism materials refine how the result may be discussed, but only within a descriptive boundary. The accepted reading is that histogram-delta features matter, a seed-bounded instability pattern repeats across the six-seed pack, and the tested lower-clip intervention is mixed and mostly harmful. The paper therefore retains descriptive mechanism support while rejecting any simple causal-closure or teacher-necessity retelling.

5. A separately labeled statistical-calibration extension lane. The same affine runtime contract can also host a simple non-neural histogram-driven calibration rule. In a separately labeled supplement-side calibration-extension analysis, that statistical-calibration path can be strong relative to the reference anchors, which supports the calibration-contract story at the systems level. However, the reported evidence remains supplementary and does not select a unique promoted threshold.

6. A layered runtime and deployment-boundary evidence chain. The repository now has one code-backed training/material regeneration pack, one clean CPU-only bounded rerun, one isolated local true `.tflite` runtime confirmation for selected preserved artifacts, and one read-only hardware-readiness pack on a host that still cannot enter board execution. These are useful supporting boundaries for the manuscript, but they remain layered evidence rather than coequal benchmark results or completed deployment claims.

2.1 Metric-level advantages

For reviewer-facing comparison, the advantages should be stated by metric and evidence layer rather than by architecture alone. Table 1 separates what is already supported by the controlled software-HIL stack from what remains architecture-level reasoning or future hardware measurement.

The table is intentionally layered. Its job is not to flatten all evidence into one success claim, but to show which advantages are already ranking-level software-HIL facts, which are extension-lane or supporting facts, and which remain hardware-facing targets.

3 Brief Review of the GKP Code

This section fixes the physical-layer notation used later in the manuscript. It does not claim a new exact GKP decoder. Its purpose is to explain why a bounded affine fast path is a useful local approximation once drift adaptation is pushed into a slower calibration loop.

3.1 Ideal and approximate code states

The square-lattice GKP code encodes a qubit into an oscillator by using periodic structure in the q and p quadratures [1]. In the convention used here, the lattice constant is

$$\lambda = \sqrt{2\pi}. \quad (4)$$

This λ is the normalized software residual coordinate used for syndrome wrapping, clipping, and half-lattice boundary tests in the benchmark. It should not be read as a new physical GKP convention or as a calibrated finite-energy device parameter. An ideal logical state may be represented heuristically as an infinite comb, for example

$$|\bar{0}\rangle \propto \sum_{n \in \mathbb{Z}} |n\lambda\rangle_q. \quad (5)$$

Ideal comb states are not physical because they require infinite energy. Approximate GKP states replace the infinitely sharp comb with finitely squeezed peaks and an envelope. Finite-energy structure is central to practical GKP QEC rather than a minor implementation detail [2]: it determines the intrinsic syndrome uncertainty and contributes an effective displacement-like noise. One useful one-quadrature picture is

$$|\tilde{0}\rangle_q \propto \sum_{n \in \mathbb{Z}} \exp\left[-\frac{(n\lambda)^2}{2\kappa^2}\right] \int dx \exp\left[-\frac{(x - n\lambda)^2}{2\Delta^2}\right] |x\rangle_q, \quad (6)$$

with an analogous comb for the conjugate quadrature. The peak width Δ represents finite squeezing, while the broad envelope scale κ keeps the state normalizable. The ideal code is recovered only in the singular limit of narrow peaks and a broad envelope. In any physical state, however, nearby lattice peaks overlap in the tails. A syndrome value therefore does not identify a displacement branch with certainty; it gives a posterior over possible lattice shifts, and that posterior becomes most ambiguous near the half-lattice boundaries.

3.2 Syndrome measurement as modular displacement information

Let the accumulated phase-space displacement error before correction be

$$e_t = \begin{bmatrix} e_{q,t} \\ e_{p,t} \end{bmatrix}. \quad (7)$$

An ideal GKP syndrome observes this displacement modulo the lattice:

$$s_t = e_t \bmod \lambda, \quad s_{q,t}, s_{p,t} \in [-\lambda/2, \lambda/2). \quad (8)$$

In a finite-energy and noisy-measurement setting, the measured syndrome is more accurately described as

$$\tilde{s}_t = \text{mod}(e_t, \lambda) + \eta_t^{\text{meas}} + \eta_t^{\text{GKP}}. \quad (9)$$

Here η_t^{meas} represents measurement and auxiliary-state contributions, while η_t^{GKP} summarizes finite-squeezing uncertainty. The decoder therefore observes a noisy modular representative, not the absolute displacement. The information in $\tilde{s}_t$ has two roles. Its signed value gives the small correction suggested inside the current fundamental cell. Its distance to the cell boundary,

$$d_{\text{bdry}}(\tilde{s}_{j,t}) = \lambda/2 - |\tilde{s}_{j,t}|, \quad j \in \{q, p\}, \quad (10)$$

indicates how fragile the branch decision is. A syndrome close to zero is locally well separated from a logical boundary, whereas a syndrome close to $\pm\lambda/2$ can flip the inferred lattice branch under small measurement, finite-squeezing, or calibration errors. This is the reason that analog GKP syndromes carry more information than binary accept/reject flags: the same measurement reports both a displacement estimate and a confidence-like proximity to a logical decision boundary.

3.3 Local affine decoding and its limits

The modulo structure makes exact GKP decoding nonlinear and branch dependent. Near a single lattice branch, however, a Gaussian approximation gives a useful local model. Fix a branch $m \in \mathbb{Z}^2$, write the local unwrapped residual as $r = e - \lambda m$, and condition on an effective noise state θ^{noise} . If r and s are jointly Gaussian within that conditioned branch,

$$\begin{bmatrix} r \\ s \end{bmatrix} \sim \mathcal{N} \left(\begin{bmatrix} \mu_r \\ \mu_s \end{bmatrix}, \begin{bmatrix} \Sigma_{rr} & \Sigma_{rs} \\ \Sigma_{sr} & \Sigma_{ss} \end{bmatrix} \right), \quad (11)$$

then the linear-MMSE estimate is

$$\hat{r} = \mu_r + \Sigma_{rs} \Sigma_{ss}^{-1} (s - \mu_s) = Ks + b, \quad (12)$$

where

$$K = \Sigma_{rs} \Sigma_{ss}^{-1}, \quad b = \mu_r - K \mu_s. \quad (13)$$

This derivation explains why an affine fast path is a plausible low-latency approximation. It also marks the boundary of the approximation. Near lattice decision boundaries, the posterior can be multimodal; maximum-likelihood, closest-lattice-point, or wrapped-Gaussian decoders can outperform a single global affine rule [3, 9]. The present framework keeps this affine approximation on the fast path and adapts it slowly through calibration of K and b , rather than claiming to solve the full wrapped posterior per shot. Equivalently, the affine rule should be read as a moment-matched correction for the branch that the runtime has already committed to after wrapping. Changes in bias, variance, or covariance alter the local conditional moments $(\mu_r, \mu_s, \Sigma_{rs}, \Sigma_{ss})$, so the best local K and b also change. The dual-loop design uses this observation as a control principle: the fast loop still applies one matrix-vector correction, but the slow loop updates the moments implicitly through teacher estimates, histogram features, or other calibration summaries. This keeps the implementation bounded while acknowledging that approximate GKP decoding is fundamentally a wrapped, noisy, branch-dependent inference problem.

3.4 Logical failure criterion

After correction, the residual displacement is wrapped into the GKP fundamental cell. A logical error occurs when the residual crosses a logical decision boundary, for example

$$|r_{q,t}| > \lambda/2 \quad \Rightarrow \quad X_L \text{ error}, \quad (14)$$

and similarly for the p -quadrature. Closed-loop logical error probability is therefore the central metric; parameter-regression error is useful only when it correlates with the downstream correction outcome. The primary benchmark therefore evaluates logical performance under the controlled software-HIL protocol, while runtime timing and hardware resource questions remain separate boundary issues.

4 Noise and Drift Model

The model below is an effective calibration model. Its role is to summarize the part of the analog GKP correction problem that the slow loop can estimate from recent syndrome statistics and map back into bounded affine runtime parameters. It is not a full circuit-level or hardware-validated noise closure.

4.1 Effective noise state

The slow loop does not attempt to infer every microscopic hardware parameter. It uses a low-dimensional effective state,

$$\theta_t^{\text{noise}} = (\sigma_t, \mu_{q,t}, \mu_{p,t}, \vartheta_t), \quad (15)$$

where σ_t controls the displacement scale, $\mu_{q,t}$ and $\mu_{p,t}$ represent mean bias, and ϑ_t captures covariance-axis rotation. This compact parameterization can be mapped to affine runtime parameters and estimated from finite histogram windows.

4.2 Noise sources represented by the model

The model is an effective displacement-noise model with additional measurement uncertainty. It absorbs several physical effects:

1. finite-energy GKP peak width and envelope effects;
2. Gaussian random displacement noise in q and p ;
3. biased displacement means caused by calibration offsets;
4. anisotropic or rotated covariance caused by asymmetric noise;
5. syndrome measurement noise and noisy auxiliary-state contributions.

This is narrower than a full circuit-level GKP noise model, but it is aligned with the affine fast-path contract. The compression is deliberate: the slow loop needs a control-oriented state that can be updated reliably from bounded windows, not an exhaustive microscopic device description.

4.3 Drift scenarios

The benchmark suite uses four drift scenarios:

1. `static_bias_theta`: a biased and rotated operating point;
2. `linear_ramp`: slowly changing effective noise parameters;
3. `step_sigma_theta`: a sudden variance/orientation change;
4. `periodic_drift`: periodic nonstationarity.

These scenarios separate different adaptation demands: steady calibration, tracking, shock response, and periodic following. They are sufficient for the current reference benchmark and extension-lane follow-ups, but they are not claimed as exhaustive drift coverage.

5 Model Architecture

The architecture is organized around one teacher-anchored residual path and several supporting or boundary layers that reuse the same affine runtime contract. Only the teacher-anchored hybrid path participates in the primary reference benchmark ranking; the statistical-calibration branch is kept as a separately labeled extension lane, and deployment-facing paths remain boundary evidence rather than completed hardware execution.

5.1 Fast loop: affine fixed-point decoder

The fast loop receives the current measured syndrome s_t , reads the active parameter bank, and computes

$$s_t^{\text{clip}} = \text{clip}(s_t, -s_{\max}, s_{\max}), \quad (16)$$

$$\Delta_t^{\text{raw}} = K_t s_t^{\text{clip}} + b_t, \quad (17)$$

$$\Delta_t = Q(\text{clip}(\Delta_t^{\text{raw}}, -\Delta_{\max}, \Delta_{\max})), \quad (18)$$

where $Q(\cdot)$ denotes fixed-point quantization. The implementation uses an FPGA-oriented Q4.20 representation for syndrome values and runtime parameters. The fast loop also records diagnostic counters, including histogram-input saturation, correction saturation, overflow, aggressive parameter events, commit counts, and fallback-related signals. The current hardware-bounded claim attaches to this fast boundary and its runtime

contract, not to completed board-level timing or resource closure. In the newer submission-draft checks, this boundary is backed only by software-facing feasibility diagnostics: the affine fast path has an analytical per-shot count of four multiplications and four additions, and Q4.20 emulation on controlled samples changed corrections by at most 1.6×10^{-6} without changing residual-boundary crossings or producing quantization saturation. These checks support the fixed-point motivation, not FPGA synthesis, timing closure, power/resource measurement, or source-vs-board agreement.

5.2 Parameter mapping

Given an estimated noise state, the parameter mapper constructs an error covariance

$$C = R(\vartheta) \begin{bmatrix} \sigma_q^2 & 0 \\ 0 & \sigma_p^2 \end{bmatrix} R(\vartheta)^\top, \quad (19)$$

and a measurement covariance

$$R_{\text{meas}} = (\sigma_{\text{meas}}^2 + \Delta_{\text{eff}}^2)I. \quad (20)$$

The raw gain is

$$K_{\text{raw}} = C(C + R_{\text{meas}})^{-1}, \quad (21)$$

followed by gain clipping or scaling. The bias target is

$$b_{\text{target}} = \alpha(I - K_{\text{target}})\mu, \quad \mu = \begin{bmatrix} \mu_q \\ \mu_p \end{bmatrix}. \quad (22)$$

Finally, exponential smoothing produces staged parameters:

$$K_t = (1 - \beta)K_{t-1} + \beta K_{\text{target}}, \quad b_t = (1 - \beta)b_{t-1} + \beta b_{\text{target}}. \quad (23)$$

This mapper defines the common committed interface shared by classical baselines, the teacher-anchored learned residual branch, and the bounded statistical-calibration extension lane.

5.3 Teacher estimators

The teacher family provides stable estimates of θ_t^{noise} from recent syndrome history. A simple teacher computes moments over a histogram window. Stronger teachers such as EKF, UKF, RLS, or particle-filter variants add state-space assumptions over time. Abstractly,

$$\hat{\theta}_t^{\text{teacher}} = \text{Teacher}(H_{1:t}), \quad (24)$$

and

$$(K_t^{\text{teacher}}, b_t^{\text{teacher}}) = \text{ParamMapper}(\hat{\theta}_t^{\text{teacher}}). \quad (25)$$

The teacher keeps the learned branch near an interpretable calibration surface and creates meaningful ablation baselines. In the controlled reference comparison, these teacher-driven baselines define the classical reference rows against which the hybrid residual branch is judged.

5.4 CNN residual branch

The CNN consumes a short context of normalized syndrome histograms and histogram deltas:

$$X_t^{\text{hist}} = [H_{t-c+1}, \dots, H_t, \Delta H_{t-c+2}, \dots, \Delta H_t]. \quad (26)$$

Teacher-side features may include

$$z_t^{\text{teacher}} = (\hat{\theta}_t^{\text{teacher}}, K_t^{\text{teacher}}, b_t^{\text{teacher}}, \Delta b_t^{\text{teacher}}, \dots). \quad (27)$$

The gated branch narrows this to a small set of teacher- b and teacher- Δb scalars. The learned branch predicts

$$\widehat{\delta b}_t = f_\phi(X_t^{\text{hist}}, z_t^{\text{teacher}}), \quad (28)$$

then clips the result:

$$\delta b_t = \text{clip}(s_b \widehat{\delta b}_t, -b_{\max}, b_{\max}). \quad (29)$$

The committed parameter is

$$K_t = K_t^{\text{teacher}}, \quad b_t = \text{EMA}(b_t^{\text{teacher}} + \delta b_t). \quad (30)$$

This matches the strongest current reference evidence: the learned winner is a teacher-anchored residual-b style branch under a bounded affine runtime contract, not an unconstrained neural decoder that replaces the fast path.

5.5 Statistical calibration branch

The same affine runtime contract also supports a non-neural statistical calibration branch. This branch estimates a bounded bias correction directly from recent syndrome statistics and emits runtime parameters through the same clipping, smoothing, and stage-and-commit path. Its role is both practical and scientific: it tests whether the benefit comes from a general histogram-driven calibration principle rather than from the CNN architecture alone. However, its status in the current manuscript remains narrow: it is a supplementary analysis, not part of the controlled reference ranking, not a promoted comparator, and not a unique-threshold result.

5.6 Stage-and-commit runtime contract

The slow loop does not directly mutate active fast-loop parameters. It stages a candidate (K, b) into an inactive bank and commits it at a safe epoch boundary:

$$(K_t, b_t) = \begin{cases} (K^A, b^A), & t < t_{\text{commit}}, \\ (K^B, b^B), & t \geq t_{\text{commit}}. \end{cases} \quad (31)$$

This contract allows stale-parameter effects, update latency, commit success, rollback/fallback behavior, and fixed-point stability to be measured separately from logical performance. Software-HIL validates that this runtime contract can be exercised and audited, while current hardware-readiness materials stop before board execution, timing and source-vs-board agreement.

6 Relationship to Existing Work

6.1 GKP and bosonic analog soft information

GKP analog syndrome information is already known to be useful. Analog-QEC, surface-GKP, and concatenated bosonic-QLDPC decoders use continuous measurement information to set decoder weights or soft messages [3, 4, 5, 6, 7, 8]. The distinction here is not the value of analog information itself, but the layer at which it is consumed: recent syndrome statistics are compressed into physical-layer affine GKP fast-path parameters rather than outer-code matching weights or LDPC messages.

6.2 Adaptive priors and syndrome-statistics estimation

Adaptive-weight and syndrome-statistics methods estimate decoder weights or noise parameters from measured data [10, 11, 12]. Calibrated decoders and decoder-prior optimization show that prior mismatch can materially affect logical performance [13, 14]. This work takes the same lesson seriously, but maps the adaptation target to the GKP affine parameters (K, b) and constrains the update through an explicit stage-and-commit runtime contract.

6.3 Learned low-latency QEC modules

AI pre-decoders, high-accuracy neural decoders, and calibration-conditioned FiLM decoders demonstrate a systems-oriented pattern: a learned module is most useful when its input/output contract, latency role, and integration with a classical decoder are explicit [15, 16, 17]. The learned module here is narrower. It is not the per-shot decoder, but a slow calibration component that updates low-dimensional affine parameters while the fast boundary remains fixed-point affine logic.

6.4 Real-time and FPGA QEC decoders

Real-time QEC decoders and FPGA-integrated systems have reported low-latency hardware decoding in stabilizer-code settings [24, 25, 26, 27]. Hardware-oriented decoder designs also include lookup-table approaches, union-find architectures, matching and greedy approximations, clustering decoders, and FPGA-tailored quantum-LDPC message-passing systems [18, 19, 20, 21, 22, 23]. Real-time bosonic QEC has also been demonstrated in experimental feedback systems [28, 29]. These works set the standard for full deployment claims: closed-loop timing, worst-case latency, resource use, and hardware integration.

The contribution here is intentionally narrower and complementary. It does not aim to replace surface-code, repetition-code, or qLDPC hardware decoders. Instead, it targets the GKP physical-layer correction problem that precedes many concatenated-code decoding settings: how should a device update the analog displacement rule when the finite-energy GKP syndrome distribution drifts? In that setting, a fixed binary graph decoder is not the natural first abstraction, and an unconstrained neural network on the per-shot path is not a compelling hardware endpoint. A staged affine parameter bank provides a smaller and more testable contract: update (K, b) slowly from recent analog statistics, then execute only $Ks + b$ at the fast boundary.

6.5 Advantages relative to existing QEC decoder approaches

The proposed architecture is best viewed as a middle layer between physical GKP syndrome acquisition and higher-level QEC decoding. Its advantages over nearby approaches are architectural and evidence-bounded rather than claims of completed board-level deployment or promoted comparator closure.

Preserving analog GKP information at the right layer. Outer-code GKP methods already show that analog syndrome values improve matching weights or belief-propagation messages. This work uses the same principle earlier in the stack: the analog histogram updates the physical GKP displacement rule itself. This can reduce prior mismatch before the information is handed to a surface-code, LDPC, or other outer decoder.

Drift adaptation without a neural real-time critical path. End-to-end neural decoders can be expressive, but they make worst-case latency, fixed-point validation, and fallback behavior harder to reason about. Here, learning is optional and narrow: a CNN residual branch or, in a separately bounded extension lane, a statistical calibration branch updates a small parameter surface, while the real-time path remains deterministic affine logic. This separation is expected to be useful when future hardware systems need both adaptation and bounded per-shot timing.

A replaceable calibration module rather than a single decoder commitment. The same runtime contract can host a teacher estimator, a learned residual branch, or a non-neural statistical calibration rule. The bounded statistical-calibration extension lane therefore supports the design story without changing the evidence hierarchy: the important object is the histogram-driven affine calibration contract, not one particular CNN architecture, and this note keeps that lane supplement-side rather than as a promoted comparator.

Explicit deployment diagnostics. The framework makes quantization, clipping, saturation, stale-parameter use, commit counts, fallback behavior, and update latency visible benchmark objects. This is

closer to FPGA-QEC practice than reporting only offline regression error, and it provides a path to future resource and timing measurements once a real hardware host is available.

Compatibility with larger QEC stacks. Because the fast path emits a physical displacement correction rather than a complete outer-code decision, the method can in principle be combined with surface-GKP, bosonic-QLDPC, or conventional stabilizer-code decoders. Its expected role is to keep the GKP layer calibrated under drift so that downstream decoders receive better-conditioned residual information.

7 Experimental Setup

7.1 Software-HIL protocol

The primary numerical experiments use a controlled software-HIL protocol that preserves the fast loop, slow loop, parameter-bank, and stage-and-commit interfaces while executing the underlying experiment in software. The authoritative reference benchmark contains four drift scenarios, five comparison modes, paired seeds, and two repeats under one fixed comparison contract. This protocol is sufficient for comparing adaptive calibration laws under matched drift trajectories and shared random structure, but it does not measure board-level latency, hardware resource use, or deployment readiness. Later ablation, multi-seed, and calibration-extension materials are therefore interpreted relative to this anchor rather than as a rewritten benchmark.

7.2 Scenarios and modes

The four drift scenarios are:

Scenario key	Interpretation
<code>static.bias.theta</code>	biased and rotated operating point
<code>linear.ramp</code>	slowly changing effective noise parameters
<code>step.sigma.theta</code>	sudden variance/orientation change
<code>periodic.drift</code>	periodic nonstationarity

The primary comparison includes five modes: EKF, UKF, a constant-mean affine baseline, an RLS residual- b baseline, and the learned hybrid residual branch. Additional bounded materials then cover feature and teacher ablations, six-seed descriptive mechanism snapshots, and a separately labeled statistical-calibration extension lane under the same affine runtime contract. The reported metric is `final_ler_mean`, where lower values indicate fewer logical failures under the fixed software-HIL protocol.

8 Numerical Results

The synchronized results are organized as an evidence ladder. The first rung is the predeclared software-HIL ranking in Table 2. The second rung adds descriptive margins and the two completed formal paired-interval scenario rows. The third rung adds controlled oracle, wrapped-Gaussian, residual-boundary, finite-squeezing, holdout and commit-lag diagnostics that explain where the affine contract works and where it fails. The final rung adds operation-count, Q4.20 fixed-point, runtime-counter and hardware-measurement rows. This ordering is deliberate: the main table supports the performance statement; the diagnostic rows explain estimator and metric limits; and the implementation rows test whether the online rule is small and observable enough for later board validation. None of these supporting rows turns the note into a hardware result, a finite-energy logical-channel fidelity estimate or an all-scenario repeat-expanded benchmark.

8.1 Four-scenario affine benchmark

Table 2 records the controlled reference benchmark that also underlies any later paper-facing visual summary. In this note, the table is kept as the primary representation because it is the authoritative numeric surface.

Under the controlled reference protocol, the hybrid residual branch is the winner in all four scenarios, with UKF as the runner-up throughout. This remains the authoritative primary result layer of the current note. It supports a controlled-simulation ranking only, and it should not be retold as an expanded benchmark, a `.tflite` ranking, or a board-level ranking.

8.2 Descriptive UKF-versus-hybrid margins

The submission draft also reports the UKF-minus-hybrid margins behind the winner labels. These rows make the main ranking auditable as a magnitude statement rather than only as a scenario-wise ordering. The hybrid branch has positive paired deltas in all available paired rows, with relative reductions from 1.75% to 2.89% against UKF in the four reference scenarios (Table 3). The average relative reduction across the tested set is 2.32%. To synchronize more of the submission result layer, the table also records the smallest observed paired delta, the non-inferential two-repeat resampling envelope, and the margin divided by the larger descriptive standard deviation of the compared rows. The latter ratio ranges from 8.05 to 27.00, but it remains an auditability diagnostic rather than a significance statistic. Because the reference anchor has only two repeats per scenario, these margins remain descriptive data readouts. They are not confidence intervals, p -values, standard errors, or a distributional robustness claim.

8.3 Completed paired-interval scenario checks

The submission draft adds a repeat-expanded paired-interval layer for the two completed formal scenario rows. In both `static_bias_theta` and `linear_ramp`, all 12 paired repeats have positive UKF-minus-hybrid `final_ler` deltas, and both the paired- t and bootstrap 95% intervals have positive lower bounds (Table 4). This supports a scenario-level positive-interval statement for these two rows only. It does not establish an all-scenario repeat-expanded advantage, pooled p -value, holdout robustness, or any hardware-facing claim.

8.4 Repeat-expansion protocol checks

The submission draft also adds execution checks for the planned repeat-expanded route. A short-run runner matrix used the same four predeclared scenarios and the UKF-versus-hybrid comparison, but with shortened timing (120 slow-loop updates and 4.8×10^5 fast cycles) and two paired repeats. All four rows completed and preserved positive UKF-minus-hybrid deltas (Table 5). This table is useful because it checks command shape, row accounting and field availability before a larger repeat budget. It is not part of the main performance evidence, and it does not convert the note into an expanded benchmark. The corresponding validation threshold is summarized in Table 6: only the reference ranking and the two formal scenario rows currently support manuscript claims, while all-scenario repeat expansion, trained-branch holdout robustness and hardware measurements remain required upgrades. The extra column in the table is synchronized from the submission draft’s validation-threshold discussion so that the note records not only what was observed, but also what additional experimental row would be needed before stronger wording could be used.

8.5 Feature and teacher ablations

Table 8 is best read as appendix-side support for the reference benchmark rather than as a replacement ranking table. Removing histogram deltas degrades performance relative to the full hybrid model, indicating that temporal changes in the syndrome histogram matter inside this fixed benchmark set. Removing teacher-side channels also changes performance materially, but the no-teacher-params row performing best means the current evidence does not support a simple teacher-necessity claim. The safest reading is therefore descriptive: feature and teacher design matter, yet the bounded ablation does not close the mechanism question.

The next compact note item preserves the accepted statistical-calibration evidence as a supplement-side extension lane rather than as a coequal Results pillar.

8.5.1 Supplement-side statistical calibration extension lane

A separately labeled extension lane adds statistical calibration variants under the same affine runtime contract. Table 9 records the bounded comparison snapshot that motivated this lane and compresses the local sensitivity findings from the submission draft. In this extension matrix, statistical calibration variants outperform UKF and the learned hybrid branch, and the signal is not tied to a single threshold point. However, the the reported evidence deliberately does not promote this lane into the main comparator set: the default and high-threshold variants remain persistently tied across aggregate and scenario-wise summaries, and the candidate-generation protocol is not matched to the main ranking. The five-mode controlled benchmark therefore remains the authoritative reference result, while the statistical-calibration lane is retained as supplement-side evidence for the broader adaptive-affine contract.

8.6 Mechanism probe for residual-b behavior

A paper-facing mechanism figure may summarize the learned residual branch, but appendix tables remain the authoritative numeric companion. The six-seed evidence is descriptive rather than causal: the instability pattern repeats broadly, while the tested lower-clip intervention is mixed and mostly harmful. This supports a cautious interpretation: residual amplitude participates in some failure modes, but it is not a monotonic causal explanation by itself, and the reported evidence does not prove teacher necessity. Appendix tables carry the bounded ablation, cross-seed snapshots, and supporting records for this conservative reading without reopening the main-text route.

The submission draft makes this caution concrete with the six-seed intervention summary in Table 10. The Gated-v5-minus-full column is often negative, indicating that the gated residual branch can improve over the full branch in several seeds. The lower-clip intervention, however, is harmful in four of six seeds and helpful in only two. The table therefore supports a mechanism-risk statement, not a successful mitigation claim.

8.7 Unseen drift generalization

The current four scenarios cover steady bias, ramp tracking, step response, and periodic drift. A stronger benchmark would require unseen drift families such as random-walk drift, $1/f$ -like drift, burst measurement noise, coupled bias-variance drift, and faster-than-window drift. That expansion remains a future benchmark lane rather than a completed claim in the present note; it is precisely the kind of follow-up needed to distinguish genuine drift adaptation from overfitting to a small set of hand-designed trajectories. The submission draft now includes a controlled non-hardware stress diagnostic for random-walk, burst/reset, and faster-than-window drift. In that diagnostic the known-state oracle affine row has the lowest residual MSE in all three stress families, while a lagged-affine reference is mixed: it approaches oracle behavior under random-walk drift but degrades under abrupt or too-fast drift. This adds a useful falsification layer for the affine contract, but it is not a trained-branch holdout benchmark, confidence-interval result, or hardware robustness result. The corresponding residual-MSE rows are summarized in Table 11. Oracle affine has the lowest residual MSE in all three stress families. Lagged affine remains below the fixed rule under random-walk drift, but becomes worse than fixed affine under burst/reset and faster-than-window oscillation. This pattern is useful because it turns stale-parameter behavior into a measurable slow-loop design variable instead of an assumed latency footnote. The oracle residual-boundary channel surrogate remains close to unity in these rows because boundary crossings are rare; it is reported only as a consistency bridge, not as a calibrated finite-energy fidelity.

The submission draft also sweeps stale-commit lag on the same holdout families. Random-walk drift degrades smoothly as lag increases, while burst/reset and faster-than-window drift are non-monotone because a stale state can occasionally align with a later reset or oscillation phase. Table 12 compresses the local-validity and lag-sweep readout: each holdout family has oracle-affine MSE headroom over the fixed rule, but stale commits can erase part of that headroom under abrupt or rapid drift. The corresponding branch-risk

deltas against oracle affine are 0.006055, 0.003792 and 0.003740 for random-walk, burst/reset and faster-than-window stress; the 32-step lag-risk deltas are 0.001182, 0.006478 and 0.005144, respectively.

8.8 Oracle and wrapped-Gaussian baselines

The affine methods still need comparison against stronger theoretical baselines: nearest-lattice hard decoding, static affine decoding, oracle affine decoding with true noise state, and wrapped-Gaussian or maximum-likelihood decoding in small settings. Those baselines would separate two kinds of loss that remain entangled here: the loss from constraining the fast path to be affine, and the loss from estimating the noise state imperfectly in the slow loop. The newer controlled local-Gaussian checks begin this separation without closing it. In one-step and short-sequence diagnostics, oracle affine improves residual MSE relative to a fixed nominal affine rule, while the direct nearest-syndrome row exposes hard-correction noise and the naive wrapped-mean or wrapped-MAP references become mixed or weaker in several drift states. The safe conclusion is not that affine decoding is globally optimal; it is that branch-aware posterior or nearest-lattice references must be implemented as tuned sequence-level baselines before they can serve as stronger comparators.

Table 13 gives the compact one-step version of that diagnostic. The direct nearest-syndrome row is much weaker in residual MSE because it applies the noisy wrapped syndrome as a hard correction. The known-state oracle affine row improves on fixed affine in all four controlled states, with the largest headroom after the step change. The wrapped posterior mean is slightly better only in the static state, and the MAP branch rule is weaker in the ramp, step and periodic states. The result strengthens the engineering rationale for calibrated affine updates, but it does not replace a formal known-noise nearest-lattice or sequence-level wrapped-decoder benchmark.

The same comparison was also run over short controlled drift sequences. This sequence-level diagnostic preserves the one-step message that oracle affine is not dominated locally, while exposing a practical risk for naive posterior branching: wrapped-mean and wrapped-MAP references give higher crossing proxies in the ramp, step, and periodic sequence states (Table 14). The result does not prove affine global optimality; it says that nearest-lattice or wrapped-posterior comparators must be implemented as tuned sequence-level decoders before they can be used as stronger baselines.

The submission draft further translates the same half-lattice event surface into a compact residual-boundary channel surrogate (Table 15). This bridge is useful because it prevents `final_1er`, Pauli-event language and fidelity language from being conflated. Across the four controlled local-Gaussian states, the fixed and oracle affine rows have the lowest mean p_{any} , while the wrapped MAP rule has the largest worst-state crossing probability. A separate finite-squeezing toy-channel check sweeps $\Delta \in \{0.18, 0.26, 0.34\}$. At the strongest toy-channel stress, the fixed/oracle affine rows remain near 1.6×10^{-4} mean p_{any} , whereas direct hard nearest-syndrome correction rises to 1.0×10^{-3} . These rows are channel-language diagnostics only; they are not finite-energy logical-channel fidelity, process tomography, hardware fidelity or outer-code logical-error estimates.

To make the synchronization less summary-like, Table 16 adds the per-state rows that are most useful for checking the channel and fixed-point claims directly. The first block reports the controlled local-Gaussian p_{any} values before aggregation: the affine rows have no observed crossings in the static and ramp states and remain below the wrapped-posterior rows in the step and periodic states. The second block reports the exact Q4.20 parity rows behind the compact datapath summary: max and p99 correction differences stay at 10^{-6} -scale, with zero crossing change and zero quantization saturation in all four tested states. These rows are still diagnostic software checks, but they expose more of the submission draft’s numerical surface inside the 25-page note.

8.9 Runtime, quantization, and fixed-point degradation

The deployment story depends on more than logical performance, so the current manuscript keeps runtime material at a supporting-boundary layer rather than flattening it into a single deployment claim. The strongest accepted runtime-facing fact is narrow but real: selected preserved float and int8 `.tflite` artifacts have been loaded and executed in one isolated local runtime environment, and that fact is strong enough to

justify an appendix-level runtime boundary table for those preserved artifacts. What remains unclosed is the broader fixed-point/runtime degradation question itself. Saturation and overflow rates under deployment-style quantization, commit latency, stale-parameter penalty, fallback frequency, host-side update cost, and source-vs-embedded drift under a default environment still require separate validation steps rather than narrative promotion. The submission draft adds two compact software diagnostics for this boundary (Table 17). First, the affine fast path requires one two-quadrature 2×2 matrix-vector product and one bias addition, which gives four multiplications, four additions, no nonlinear operations, and six stored state scalars. By contrast, the one-step 3×3 wrapped-branch references have much larger arithmetic and state footprints. Second, Q4.20 software fixed-point emulation on controlled local-Gaussian samples produces only 10^{-6} -scale correction differences, no crossing delta, and no observed quantization saturation in the tested rows. These facts support a fixed-point feasibility argument for the affine datapath. They still do not establish FPGA synthesis, timing closure, resource use, power, or source-vs-board agreement.

The same software run records comparator-level runtime counters (Table 18). All five modes apply about 900 commits, report zero slow-update violations and zero correction saturation, and keep overflow near 2.5×10^{-3} . These counters make the stage-and-commit contract observable in software; they do not measure board latency, rollback reliability or source-vs-board agreement.

8.10 Embedded runtime and board-level validation

Embedded runtime and board-level validation remain split from the simulation results. The local-runtime layer only states that selected preserved `.tflite` artifacts execute in one isolated software environment. The hardware layer remains a measurement target because no board timing, resource, power or source-vs-board row is reported. Table 19 therefore synchronizes the submission draft’s statistical-treatment, runner-coverage, source-data and hardware-validation status without turning requirements into results.

8.11 Analysis-file coverage and statistical treatment

The expanded result layer is only useful if the reader can tell which numbers come from checked analysis files and which rows are protocol or requirement statements. Table 20 synchronizes this traceability layer from the submission draft. The main benchmark remains a descriptive paired-seed comparison with two repeats per scenario; the completed 12-pair rows are scenario-level interval checks; and the diagnostic tables are used to test mechanism, local validity, fixed-point feasibility and stale-commit risk rather than to establish a new leaderboard.

8.12 Hardware measurement requirements synchronized from the submission draft

The present note does not contain real-board execution results. The available deployment-adjacent material supports only one narrow runtime statement: selected preserved `.tflite` artifacts execute in an isolated software runtime. Board execution, timing, resource, power and source-vs-board measurements remain outside the synchronized result set. The hardware-facing requirements in Table 21 are therefore included as a validation surface rather than as a result table.

9 Discussion

The results and architecture support a narrower claim than “a CNN is a universal GKP decoder”. The defensible claim is that drift-adaptive GKP correction can be organized as low-dimensional affine calibration: recent syndrome statistics update (K, b) , while the per-shot correction remains a deterministic fixed-point rule. This makes the teacher-anchored CNN branch, the statistical-calibration branch and the filtering baselines different answers to the same runtime question, not unrelated decoder families. It also separates the work from binary FPGA surface-code decoders, outer-code GKP soft-information decoders and end-to-end neural QEC models: the learned or statistical module updates a physical displacement surface, but it is not placed on the latency-critical correction path.

The advantage should be interpreted at two levels. At the primary benchmark level, the strongest result is still the controlled four-scenario software-HIL ranking: the teacher-anchored hybrid branch beats the filtering baselines across all reference scenarios. At the model-comparison level, the reported evidence does not require claiming that the CNN is always the best estimator. A separately labeled calibration-extension supplement is instead useful because it shows that the same runtime contract can host a strong non-neural alternative; however, the evidence keeps that analysis supplement-side because the default and high-threshold settings remain persistently tied. The learned branch therefore remains valuable as a replaceable slow-loop module rather than as a uniquely validated final comparator.

The systems-level advantage is modularity under real-time constraints. A future device could replace the slow estimator, teacher policy or fallback logic while preserving the affine fast path, or tighten the fast-path timing without retraining the estimator. The architecture therefore trades estimator sophistication against update cadence without putting neural inference or a graph decoder on the per-shot critical path.

The synchronized result layer should still be read in tiers. The reference ranking and two completed paired-interval scenarios are the strongest performance evidence. The ablation, statistical-calibration, controlled oracle/wrapped, channel-surrogate, holdout, fixed-point and runtime-counter rows explain where the affine contract works, where it fails and what remains measurable. They do not convert the study into a hardware result. Any claim about board latency, resources, power, deployment portability or source-vs-board agreement still requires a future `Linux + FPGA` measurement path.

10 Conclusion

This work frames drift-adaptive GKP correction as a dual-loop affine calibration problem. The fast loop remains a bounded fixed-point rule, $\Delta_t = K_t s_t + b_t$, while a slower loop updates the affine parameters from recent syndrome statistics. The formulation creates a practical middle ground between fixed decoders and full neural decoders on a deterministic per-shot path.

The numerical results support two main conclusions. First, under the controlled four-scenario software-HIL suite, adaptive affine decoding with a teacher-anchored residual branch improves over filtering baselines, with UKF-relative mean reductions of 1.75%–2.89% in the four reference rows and two completed 12/12-positive interval checks, while bounded ablations and multi-seed mechanism materials show that histogram-delta features carry useful information and that teacher-side inputs do not admit a simple necessity story. Second, a supplementary statistical-calibration analysis shows that simple non-neural estimators can also be strong under the same runtime contract, but the reported evidence does not select a unique promoted threshold. Together, these results suggest that the strongest scientific story is not that one CNN architecture has closed the comparison, but that low-dimensional histogram-driven calibration of an affine GKP fast path is a promising structure for drift-adaptive decoding.

The note now synchronizes the submission draft’s main experimental hierarchy: four-scenario ranking, two completed paired-interval rows, controlled oracle and wrapped-Gaussian diagnostics, residual-boundary channel surrogates, holdout and lag stress tests, operation counts, fixed-point emulation and runtime counters. The remaining gaps are equally explicit: all-scenario repeat-expanded intervals, stronger sequence-level baselines, default-environment `.tfLite` portability and board-level timing or resource claims still require separate validation. The conclusion is therefore an evidence-synchronized theory note with clear hardware and portability limits, not deployment validation.

References

- [1] D. Gottesman, A. Kitaev, and J. Preskill, “Encoding a qubit in an oscillator,” *Physical Review A*, 2001.
- [2] A. L. Grimsmo and S. Puri, “Quantum Error Correction with the Gottesman–Kitaev–Preskill Code,” *PRX Quantum* 2, 020101, 2021.

- [3] K. Fukui, A. Tomita, A. Okamoto, and K. Fujii, “High-threshold fault-tolerant quantum computation with analog quantum error correction,” *Physical Review X*, 2018.
- [4] K. Noh and C. Chamberland, “Fault-tolerant bosonic quantum error correction with the surface-GKP code,” *Physical Review A*, 2020.
- [5] K. Noh, C. Chamberland, and F. G. S. L. Brandao, “Low-overhead fault-tolerant quantum error correction with the surface-GKP code,” *PRX Quantum*, 2022.
- [6] N. Raveendran, N. Rengaswamy, F. Rozpedek, A. Raina, L. Jiang, and B. Vasic, “Finite-rate QLDPC-GKP coding scheme that surpasses the CSS Hamming bound,” *Quantum*, 2022.
- [7] L. Berent et al., “Analog information decoding of bosonic quantum LDPC codes,” *PRX Quantum*, 2024.
- [8] S. K. Borah, A. K. Pradhan, N. Raveendran, M. Pacenti, and B. Vasic, “Fault tolerant decoding of QLDPC-GKP codes with circuit level soft information,” preprint, 2025.
- [9] M.-A. Roy, T. Pousset, and B. Royer, “Decoding multimode Gottesman–Kitaev–Preskill codes with noisy auxiliary states,” preprint, 2025.
- [10] S. Spitz, B. Tarasinski, C. W. J. Beenakker, and T. E. O’Brien, “Adaptive weight estimator for quantum error correction,” *Advanced Quantum Technologies*, 2018.
- [11] T. Wagner, H. Kampermann, and D. Bruß, “Optimal noise estimation from syndrome statistics of quantum codes,” *Physical Review Research*, 2021.
- [12] H. Chen et al., “DGR: Tackling drifted and correlated noise in quantum error correction via decoding graph re-weighting,” preprint, 2023.
- [13] E. H. Chen et al., “Calibrated decoders for experimental quantum error correction,” *Physical Review Letters*, 2022.
- [14] V. Sivak, M. Newman, and P. Klimov, “Optimization of decoder priors for accurate quantum error correction,” *Physical Review Letters* 133, 150603, 2024.
- [15] C. Chamberland, J. Olle, M. Li, S. Thornton, and I. Baratta, “Fast and accurate AI-based pre-decoders for surface codes,” arXiv:2604.12841, 2026.
- [16] S. Stein, S. Kan, C. Liu, A. Harkness, S. Garner, Z. Du, Y. Ding, Y. Mao, and A. Li, “Calibration-conditioned FiLM decoders for low-latency decoding of quantum error correction evaluated on IBM repetition-code experiments,” preprint, 2026.
- [17] Google Quantum AI and collaborators, “Learning high-accuracy error decoding for quantum processors,” preprint, 2023.
- [18] “LILLIPUT: A lightweight low-latency lookup-table based decoder for near-term quantum error correction,” *ASPLOS*, 2022.
- [19] “AFS: Accurate, fast, and scalable error-decoding for fault-tolerant quantum computers,” *HPCA*, 2022.
- [20] “Scalable quantum error correction for surface codes using FPGA,” preprint, 2023.
- [21] “Astrea: Accurate quantum error-decoding via practical minimum-weight perfect-matching acceleration,” *ISCA*, 2023.
- [22] “A real-time, scalable, fast and resource-efficient decoder for a quantum computer,” *Nature Electronics*, 2025.

- [23] “FPGA-tailored algorithms for real-time decoding of quantum LDPC codes,” preprint, 2025.
- [24] L. Caune et al., “Demonstrating real-time and low-latency quantum error correction with superconducting qubits,” preprint, 2024.
- [25] X. Yang et al., “Real-time surface-code error correction using an FPGA-based neural-network decoder,” preprint, 2026.
- [26] M. Ziad et al., “Local clustering decoder: a fast and adaptive hardware decoder for the surface code,” preprint, 2024.
- [27] L. Maurer et al., “Real-time decoding of the gross code memory with FPGAs,” preprint, 2025.
- [28] V. Sivak et al., “Real-time quantum error correction beyond break-even,” *Nature*, 2023.
- [29] D. Lachance-Quirion et al., “Autonomous quantum error correction of Gottesman–Kitaev–Preskill states,” *Physical Review Letters*, 2024.

Metric	Advantage	Architectural reason	Supported scope
Logical-error proxy	Lower error on the controlled software-HIL benchmark than the five reference baselines; the separately labeled statistical-calibration extension lane can also outperform the reference anchors under its bounded protocol.	Both learned and statistical branches update the same physical-layer affine surface, so adaptation acts directly on the displacement rule used by the GKP fast loop.	Supported for the controlled reference ranking and the supplementary statistical-calibration analysis; not a paper-grade expanded benchmark and not a promoted comparator.
Per-shot latency	Low latency by design on the correction path.	The fast loop evaluates only a 2×2 affine map, bias addition, clipping, quantization, and parameter-bank selection; CNN or statistical estimation is kept off the per-shot critical path.	Architectural claim; board timing and worst-case latency remain a hardware measurement target.
FPGA resource and cost	Expected small fast-path resource footprint and lower verification burden than placing a neural or graph decoder in the real-time path.	The real-time datapath needs only fixed-point multiply-adds, saturation logic, and a compact parameter bank; the slow estimator can run less frequently on a host, embedded processor, or offline update service.	Resource use and energy/cost are not yet measured on a real device.
Engineering implementation	The control boundary is simple to integrate, test, and fail safely.	Quantized (K, b) , clipping limits, stale-parameter handling, saturation counters, and double-buffered stage-and-commit updates form explicit runtime contracts.	Software-HIL validates the interface shape; board execution, source-vs-board agreement and hardware timing remain unavailable in the present evidence.
Drift robustness	Adaptation can track mean, variance, and orientation changes in the effective GKP noise state.	Histogram windows and teacher/statistical summaries update the affine parameters over time instead of freezing one calibration point.	Supported for the current four-scenario effective-drift suite; broader drift families and holdout protocols remain benchmark-expansion targets.
Modularity	The best slow-loop estimator can change without changing the fast path.	Teacher residual CNN, statistical calibration, temporal models, and gain-scheduled parameter banks all map to the same committed (K, b) interface.	Current evidence supports this mainly through the teacher-anchored hybrid path and the supplement-side statistical-calibration extension lane; other slow-loop variants remain candidates for later validation.
Compatibility with larger QEC stacks	The method can serve as a GKP physical-layer calibration module before an outer stabilizer, surface-GKP, or LDPC-GKP decoder.	It improves the physical displacement correction and residual analog statistics instead of replacing the outer-code decoder decision rule.	System-level concatenated-code gains remain unmeasured.

Table 1: Metric-level advantage summary. The strongest reported evidence is logical-error reduction in software-HIL, with ‘statcalib’ retained only as a separately labeled extension-lane supplement. Latency, resource, and cost advantages are architecture-level consequences of the affine fast path and should be measured on hardware before being written as deployment results.

Scenario	EKF	UKF	Const.- μ	RLS- b	Hybrid- b
<code>static.bias.theta</code>	0.838110	0.825370	0.836658	0.837577	0.810902
<code>linear.ramp</code>	0.819200	0.811201	0.816911	0.819373	0.787755
<code>step.sigma.theta</code>	0.822365	0.811548	0.819784	0.821493	0.788800
<code>periodic.drift</code>	0.832192	0.821558	0.829670	0.832334	0.806392

Table 2: Four-scenario affine benchmark. Entries are `final_ler_mean`; lower is better.

Scenario	Mean Δ	Rel. red.	Min Δ	Envelope low	Envelope high	Δ /max SD
<code>static_bias_theta</code>	0.014469	1.75%	0.013953	0.013953	0.014984	12.17
<code>linear_ramp</code>	0.023446	2.89%	0.023017	0.023017	0.023875	27.00
<code>step_sigma_theta</code>	0.022748	2.80%	0.022440	0.022440	0.023056	21.28
<code>periodic_drift</code>	0.015166	1.85%	0.013571	0.013571	0.016762	8.05
<code>all_scenarios</code>	0.018957	—	—	0.016044	0.021892	—

Table 3: Descriptive UKF-minus-hybrid margins and two-repeat resampling envelope in the reference benchmark. Δ is UKF `final_ler` minus Hybrid-*b* `final_ler`. Positive values indicate lower hybrid error; all four scenario rows have 2/2 positive paired deltas. The envelope and final scale column are descriptive data readouts, not confidence intervals, *p*-values, standard errors, robustness proof or expanded-benchmark evidence.

Scenario	<i>n</i>	Mean Δ	SD	Min-max Δ	95% intervals
<code>static_bias_theta</code>	12	0.015563	0.001236	[0.013953, 0.018355]	[0.014778, 0.016348]; boot. [0.014933, 0.016256]
<code>linear_ramp</code>	12	0.022417	0.001892	[0.019951, 0.024946]	[0.021215, 0.023619]; boot. [0.021405, 0.023440]

Table 4: Completed scenario-level UKF-minus-hybrid interval checks. The rows summarize the completed formal-length paired repeats for two scenarios only; all 12/12 paired deltas are positive in both rows, but the table does not complete the all-scenario validation set.

Scenario	UKF	Hybrid- <i>b</i>	Δ	Rel. red.
<code>static_bias_theta</code>	0.817940	0.801012	0.016927	2.07%
<code>linear_ramp</code>	0.840509	0.813909	0.026600	3.16%
<code>step_sigma_theta</code>	0.828792	0.816023	0.012769	1.54%
<code>periodic_drift</code>	0.810660	0.781844	0.028817	3.55%

Table 5: Short-run protocol check for the planned repeat-expanded anchor comparison. Entries are not formal benchmark rows; they verify runner coverage and missing-row accounting for the expansion route only.

Layer	Synchronized result readout	Supported wording in this note	Upgrade needed for stronger wording
Reference ranking	Four scenarios, five modes, paired seeds and two repeats; Hybrid- <i>b</i> has the lowest mean <code>final_ler</code> in all four rows, with UKF as runner-up.	Descriptive software-HIL ranking only.	More repeats, formal intervals, stronger tuned decoder baselines and holdout drift rows.
Paired-repeat evidence	Short-run all-scenario UKF/Hybrid rows preserve positive deltas; two completed formal rows have 12/12 positive pairs and positive paired- <i>t</i> /bootstrap lower bounds.	Scenario-level positive interval statements for <code>static_bias_theta</code> and <code>linear_ramp</code> only.	Complete the other two scenarios with at least 12 paired repeats, preferably 16, plus pooled all-scenario analysis.
Mechanism and estimator checks	Histogram-delta removal is harmful, teacher feature removal is non-monotone, the lower-clip intervention is harmful in four of six seeds, and the statistical-calibration extension gives <code>final_ler</code> $\approx$ 0.4317–0.4671 under a non-matched protocol.	Feature sensitivity and affine-interface evidence, not causal closure or a promoted comparator.	Matched ablation repeats and a predeclared intervention test that improves rather than degrades the residual branch.
Controlled decoder/stress diagnostics	Oracle affine improves residual MSE in all controlled local states and holdout families; lagged affine fails under burst/reset and faster-than-window stress, and wrapped-Gaussian references are mixed.	Local-validity and stale-commit diagnostics, not trained-branch holdout robustness or a tuned nearest-lattice benchmark.	Train/evaluate on predeclared unseen drift families and add tuned nearest-lattice or sequence-level wrapped-posterior baselines.
Datapath and runtime checks	The affine path uses four multiplications and four additions; Q4.20 emulation changes corrections at the 10^{-6} scale with no crossing changes, and runtime counters record $\sim$ 900 commits per mode with zero slow violations.	Fixed-point/software observability evidence, not FPGA timing, resource, power, source-vs-board or board-execution evidence.	Board logs, bitstream/RTL hash, latency/resource/power measurements and source-vs-board vectors.

Table 6: Compressed result-layer map for the synchronized experimental material. The table brings more of the submission-draft result hierarchy into the note while preserving the validation boundary for all-scenario, holdout and board claims.

Submission result layer	Additional numerical content synchronized here	Compact readout	Boundary retained
Main software-HIL results	Four-scenario ranking, UKF-minus-hybrid margins, two completed paired-interval rows and a short-run expansion check.	Hybrid wins 4/4 scenarios; relative reduction 1.75%–2.89%; two formal rows have 12/12 positive pairs.	Descriptive ranking; only two completed 12-pair interval rows.
Ablation and mechanism panel	Feature/teacher ablations and six-seed lower-clip intervention readout.	No-hist-delta row degrades to 0.826723; lower-clip intervention is harmful in four of six seeds.	Feature sensitivity; no teacher-necessity or mitigation claim.
Statistical-calibration panel	Separate non-neural calibration lane under the same affine runtime contract.	Best extension rows span 0.431708–0.467083; default and high-threshold aggregates remain tied at about 0.44925.	Supplement-side extension only; no promoted unique threshold and no matched main-table comparator claim.
Controlled decoder and stress diagnostics	Oracle/wrapped-Gaussian checks, holdout drift stress and stale-commit sweeps from the submission draft.	Oracle improves controlled residual MSE in all local states; holdout oracle gains are 6.74%–7.84%, while 32-step lag risk reaches 0.006478.	Diagnostic only; no trained-branch holdout proof.
Channel and finite-squeezing bridge	Residual-boundary channel summary plus finite-squeezing toy-channel rows.	Fixed/oracle mean $p_{any} = 1.27 \times 10^{-4}$; wrapped-MAP mean 2.129×10^{-3} ; at $\Delta = 0.34$, hard nearest is 1.002×10^{-3} vs fixed affine 1.63×10^{-4} .	Surrogate channel language only; no fidelity claim.
Datapath and runtime panel	Analytical operation counts, Q4.20 emulation, software runtime counters and hardware-measurement requirements.	Affine path uses 4 multiplications, 4 additions and 6 scalars; Q4.20 max diff $\leq 2 \times 10^{-6}$; runtime counters show ~ 900 commits and zero slow violations.	Software observability; no FPGA timing, resource, power or board result.

Table 7: Numerically expanded synchronization map for the submission-draft result layer. The note does not reproduce the larger rendered figures, but it now imports the main quantitative anchors behind the performance, ablation, calibration, stress, surrogate-channel and runtime panels in compact form.

Mode	Avg. LER	Δ vs UKF	Δ vs Hybrid Full
ukf	0.817382	0.000000	+0.018837
hybrid_full	0.798545	-0.018837	0.000000
hybrid_no_hist_deltas	0.826723	+0.009341	+0.028178
hybrid_no_teacher_prediction	0.807251	-0.010131	+0.008706
hybrid_no_teacher_params	0.749621	-0.067761	-0.048924
hybrid_no_teacher_deltas	0.800329	-0.017053	+0.001784

Table 8: Feature and teacher ablation summary. Negative Δ means improvement over the reference.

Row	UKF	Hybrid-b	StatCalib value	Extension readout
static_bias_theta	0.825370	0.810902	0.431708	best variant: high-threshold; gap 0.379889
linear_ramp	0.811201	0.787755	0.467083	best variant: default; gap 0.321178
step_sigma_theta	0.811548	0.788800	0.460016	best variant: default; gap 0.328883
periodic_drift	0.821558	0.806392	0.438751	best variant: default; gap 0.367232
default variant	–	–	0.449254	mean rank 1.25; 3 scenario wins
high-threshold variant	–	–	0.449241	mean rank 1.75; 1 scenario win

Table 9: Compressed statistical-calibration extension-lane summary. Scenario rows report **final_ler_mean**; lower is better. Gap is the hybrid residual LER minus the best statistical-calibration LER in the bounded extension lane. The final two rows summarize the local sensitivity tie between default and high-threshold settings. Because this protocol is not matched to the main ranking and does not identify a unique promoted setting, the table supports supplement-side affine-calibration evidence only.

Seed	Gated v5 – Full	I1 – Gated v5	Verdict
20260425	0.000907	0.163289	harmful
20260427	-0.145352	0.287166	harmful
20260428	-0.078998	0.057395	harmful
20260429	-0.127948	0.322245	harmful
20260430	-0.170777	-0.024372	mixed/no clear effect
20260510	-0.003953	-0.035533	helpful

Table 10: Descriptive six-seed mechanism and intervention summary synchronized from the submission draft. The lower-clip intervention does not identify a causal mechanism and should not be described as a mitigation.

Holdout family	Fixed	Lagged	Oracle	Wrapped mean	Wrapped MAP	Oracle F_{avg}^{surr}
<code>random_walk</code>	0.078869	0.073867	0.072685	0.078740	0.080857	0.999915
<code>burst_reset</code>	0.066865	0.068622	0.062144	0.065936	0.067789	0.999780
<code>faster_window</code>	0.068354	0.068890	0.063745	0.067485	0.068764	0.999969

Table 11: Controlled known-state holdout drift stress diagnostic. Entries are residual MSE across 384 sequences of 512 steps except for the final surrogate column; lower residual MSE is better. The lagged row is a known-state slow-commit reference, not the trained residual branch. The surrogate column is induced by the oracle-affine residual-boundary identity rate and is not a finite-energy logical-channel fidelity. The table is a non-hardware stress test, not holdout robustness proof.

Holdout family	Oracle gain	Lag 0	Lag 8	Lag 16	Lag 32	Lag 64	Lag 128	Readout
<code>random_walk</code>	7.84%	0.073831	0.073976	0.074130	0.074410	0.075173	0.076093	lag hurts smoothly
<code>burst_reset</code>	7.06%	0.067454	0.068193	0.068657	0.068694	0.068583	0.066913	reset timing matters
<code>faster_window</code>	6.74%	0.069251	0.069406	0.068989	0.069039	0.068842	0.068249	phase alignment matters

Table 12: Compressed affine local-validity and commit-lag diagnostic. Oracle gain is the residual-MSE reduction of oracle affine relative to fixed affine. Lag columns sweep additional simulation-step lag for lagged-affine residual MSE while the commit interval is fixed at 64 simulation steps. This is a known-state slow-loop diagnostic, not trained-branch holdout proof or board-latency evidence.

Controlled state	Nearest	Fixed	Oracle	Wrapped mean	Wrapped MAP
<code>static.bias.theta</code>	0.232823	0.049199	0.048095	0.047955	0.048203
<code>linear_ramp_midpoint</code>	0.230689	0.061417	0.060251	0.062341	0.063105
<code>step_after_jump</code>	0.233626	0.108216	0.092541	0.105815	0.110339
<code>periodic_high_phase</code>	0.231307	0.081757	0.076028	0.082349	0.084002

Table 13: Controlled one-step local-Gaussian residual-MSE diagnostic. Lower is better. The nearest row is a direct hard-correction reference, not a tuned finite-energy nearest-lattice decoder. This is a sanity check for estimator and decoder limits, not a formal software-HIL or hardware benchmark.

Scenario	Nearest	Fixed	Oracle	Wrapped mean	Wrapped MAP
<code>static.bias</code>	0.000214	0.000000	0.000000	0.000000	0.000000
<code>linear_ramp</code>	0.000264	0.000015	0.000015	0.000158	0.000575
<code>step.sigma</code>	0.000580	0.000310	0.000310	0.001221	0.003220
<code>periodic</code>	0.000229	0.000005	0.000005	0.000127	0.000519

Table 14: Sequence-level controlled baseline diagnostic. Entries are mean half-lattice residual-boundary crossing proxies across 384 sequences of 512 steps; lower is better. The nearest row is a direct hard-correction stress reference, not a tuned nearest-lattice decoder.

Diagnostic	Method	Mean p_{any}	Worst p_{any}	Mean $F_{\text{avg}}^{\text{sur}}$
Boundary surrogate	Fixed affine	0.000127	0.000467	0.999915
Boundary surrogate	Oracle affine	0.000127	0.000467	0.999915
Boundary surrogate	Wrapped mean	0.000769	0.002417	0.999488
Boundary surrogate	Wrapped MAP	0.002129	0.006450	0.998581
Toy channel, $\Delta = 0.18$	Hard nearest	0.000148	0.000517	0.999901
Toy channel, $\Delta = 0.18$	Fixed affine	0.000148	0.000517	0.999901
Toy channel, $\Delta = 0.18$	Oracle affine	0.000148	0.000517	0.999901
Toy channel, $\Delta = 0.26$	Hard nearest	0.000160	0.000475	0.999893
Toy channel, $\Delta = 0.26$	Fixed affine	0.000129	0.000458	0.999914
Toy channel, $\Delta = 0.26$	Oracle affine	0.000129	0.000458	0.999914
Toy channel, $\Delta = 0.34$	Hard nearest	0.001002	0.001500	0.999332
Toy channel, $\Delta = 0.34$	Fixed affine	0.000163	0.000617	0.999892
Toy channel, $\Delta = 0.34$	Oracle affine	0.000165	0.000625	0.999890

Table 15: Compressed residual-boundary channel bridge synchronized from the submission draft. p_{any} is the probability of at least one q/p half-lattice residual crossing. $F_{\text{avg}}^{\text{sur}}$ is the Pauli-style surrogate induced by the no-crossing probability. The rows make the metric hierarchy auditable, but they are not calibrated finite-energy GKP logical-channel fidelity, process fidelity or hardware evidence.

Block	State	Col. 1	Col. 2	Col. 3	Col. 4
<i>Residual-boundary channel block: Col. 1–4 are Fixed affine, Oracle affine, Wrapped mean and Wrapped MAP p_{any}.</i>					
Channel	<code>static.bias</code>	0.000000	0.000000	0.000000	0.000017
Channel	<code>linear.ramp</code>	0.000000	0.000000	0.000042	0.000183
Channel	<code>step.after.jump</code>	0.000467	0.000467	0.002417	0.006450
Channel	<code>periodic.high</code>	0.000042	0.000042	0.000617	0.001867
<i>Fixed-point block: Col. 1–4 are max diff, p99 diff, MSE delta and crossing delta; quantization saturation is zero in all rows.</i>					
Q4.20	<code>static.bias</code>	1.0×10^{-6}	1.0×10^{-6}	0.000000	0.000000
Q4.20	<code>linear.ramp</code>	1.0×10^{-6}	1.0×10^{-6}	0.000000	0.000000
Q4.20	<code>step.after.jump</code>	2.0×10^{-6}	1.0×10^{-6}	0.000000	0.000000
Q4.20	<code>periodic.high</code>	1.0×10^{-6}	1.0×10^{-6}	0.000000	0.000000

Table 16: Per-state residual-boundary and fixed-point rows synchronized from the submission draft. The channel block is a controlled local-Gaussian diagnostic, not finite-energy logical-channel tomography. The Q4.20 block is software fixed-point emulation only, not source-vs-board agreement, synthesis, timing, resource or power evidence.

Diagnostic row	Mult.	Add.	Nonlinear ops	Key numerical check
Affine fast path	4	4	0	six state scalars
Wrapped MAP	49	40	0	3×3 branch search
Wrapped mean	99	98	18	posterior-weighted mean
Q4.20 static	–	–	–	max diff 1.0×10^{-6} ; no crossing change
Q4.20 ramp	–	–	–	max diff 1.0×10^{-6} ; no crossing change
Q4.20 step	–	–	–	max diff 2.0×10^{-6} ; no crossing change
Q4.20 periodic	–	–	–	max diff 1.0×10^{-6} ; no crossing change
Runtime counters	–	–	–	commits/mode ~ 900 ; slow violation 0; overflow ~ 0.0025

Table 17: Compressed cost and fixed-point diagnostics for the affine fast path. The arithmetic rows are analytical per-shot counts. The fixed-point row is software emulation only, and the runtime row is a software protocol counter summary. The table supports the bounded datapath rationale, not measured FPGA timing, resources, power, or source-vs-board agreement.

Mode	Commits	Slow viol.	Fast viol.	Overflow	Corr. sat.
Const.- μ	899.8	0	0.0000158	0.002582	0
EKF	899.8	0	0.0000158	0.002559	0
Hybrid- b	899.9	0	0.0000158	0.002536	0
RLS- b	899.8	0	0.0000158	0.002538	0
UKF	899.8	0	0.0000158	0.002571	0

Table 18: Software runtime-discipline counters averaged across scenario-level comparison rows. The table supports software observability of the runtime contract only; it is not board commit latency, source-vs-board agreement, hardware reliability or FPGA timing evidence.

Layer	Synchronized experimental or reporting status	Missing for a stronger claim
Main ranking and descriptive uncertainty	Four scenarios, five modes, paired seeds and two repeats; Hybrid- <i>b</i> has the lowest mean <code>final_ler</code> in all four rows. The UKF-minus-hybrid margins are positive in all paired rows, with relative reductions of 1.75%–2.89% and a two-repeat resampling envelope reported only as a descriptive readout.	More repeats, formal confidence intervals, predeclared tests and stronger tuned decoder baselines.
Completed interval rows	<code>static.bias.theta</code> and <code>linear_ramp</code> have 12/12 positive UKF-minus-hybrid paired deltas with positive paired- <i>t</i> and bootstrap lower bounds ([0.014778, 0.016348], [0.014933, 0.016256]; [0.021215, 0.023619], [0.021405, 0.023440]).	The other two reference scenarios and pooled all-scenario inference remain incomplete.
Runner and row-accounting check	A shortened four-scenario UKF-versus-hybrid run with two paired repeats completed all requested rows and preserved positive deltas while exposing the required fields for the expansion route.	It is an execution-coverage check, not manuscript performance evidence, an interval or a holdout result.
Diagnostic experiment layers	Controlled oracle/wrapped-Gaussian rows, sequence-level checks, holdout stress tests, commit-lag sweeps and residual-boundary and finite-squeezing channel surrogates make estimator, branch and stale-commit limits auditable.	These diagnostics are not trained-branch holdout proof, calibrated finite-energy logical-channel fidelity, process tomography or hardware robustness evidence.
Implementation feasibility	The affine row uses 4 multiplications, 4 additions and 6 state scalars; Q4.20 emulation on controlled samples shows 10^{-6} -scale correction differences with no crossing change or quantization saturation; runtime counters show ~ 900 commits per mode with zero slow violations.	FPGA synthesis, timing, resource, power, source-vs-board agreement and board commit latency.
Runtime and supporting records	Selected preserved <code>.tfllite</code> artifacts execute in one isolated runtime; source tables, figure-generation records, row-level scenario/mode/seed metadata and file-level checksums support the manuscript tables.	Default-runtime portability, full independent reproduction, containerized reruns and deployment readiness.
Hardware-validation surface	A hardware-stage run would need host identity, device access, bitstream/RTL hash, AXI/DMA contract, latency distribution, commit/rollback counters, resource, power and source-vs-board vectors.	No real-board execution, timing, resource, power or source-vs-board measurement is reported here.

Table 19: Synchronized validation-status and reproducibility matrix for the expanded result layer. The table imports the submission draft’s statistical treatment, runner coverage, diagnostic-experiment, runtime-record and hardware-validation status into the note. It is a compact reporting aid, not an additional experiment, independent reproduction package or hardware claim.

Coverage group	Synchronized checked material	Interpretation limit
Main performance tables	Main result, paired-delta, two-repeat resampling envelope and advantage-margin rows are checked against the underlying comparison data.	Descriptive software comparison only; not a confidence interval, standard error, p -value or expanded benchmark.
Completed interval rows	The <code>static_bias_theta</code> and <code>linear_ramp</code> formal paired rows are generated from the repeat summary and checked against the interval table.	Two completed scenarios only; not pooled or all-scenario inference.
Controlled diagnostics	Oracle-affine, wrapped-Gaussian, sequence-baseline, holdout-stress, commit-lag, residual-boundary and finite-squeezing toy-channel rows are checked against generated CSV tables.	Diagnostic support only; not trained-branch holdout proof or calibrated logical-channel fidelity.
Implementation feasibility	Fast-path arithmetic, Q4.20 fixed-point parity, runtime-discipline counters and validation-contract rows are tied to generated analysis tables or figure data.	Analytical and software-emulation evidence only; not FPGA synthesis, timing, resource, power or source-vs-board agreement.
Analysis and reporting maps	Figure manifests, row metadata, file-level analysis manifests, metric-readiness and literature crosswalks support table and citation consistency.	Traceability aid only; not a full independent reproduction package or normalized external leaderboard.
Validation protocols	Runner-smoke, benchmark-expansion, availability, runtime-record and hardware-plan rows are classified separately from measured results.	Protocol or requirement material only; not performance evidence or hardware validation.

Table 20: Compressed analysis-file coverage and statistical-treatment matrix for the synchronized results. It identifies the submission-draft result families imported into this note and preserves the boundary between checked analysis rows, descriptive statistics, diagnostic rows and hardware requirements.

Hardware-facing item	Current treatment	Evidence needed for hardware claims
Board platform and host	Not measured in this study	Linux + FPGA host, device path, permissions, board model and driver version.
Bitstream / RTL / DMA interface	Not measured in this study	Bitstream hash, AXI map, DMA histogram shape, element width and timeout policy.
Fast-path latency	Not measured in this study	p50/p95/p99 and worst-case cycles for $Ks + b$, clipping and bank selection.
Commit latency and reliability	Not measured in this study	Stage, commit, acknowledgement, rollback and stale-parameter counters.
Resource and power	Not measured in this study	LUT/FF/DSP/BRAM, clock frequency, timing closure and power.
Source-vs-board agreement	Not measured in this study	Fixed-point agreement between the software reference and board output on shared test vectors.

Table 21: Hardware-measurement requirements synchronized from the submission draft. The entries define the implementation-stage evidence required before hardware claims can be made; they are not results of the present note.